

LLM-Based Intelligent Notification Composition

Reference Implementation: iOS SDK, Python Backend, and Sample App

Nilesh Agrawal

nilesh.d.agrawal@gmail.com • Seattle, WA

Swift 5.9

Python 3.11

FastAPI

SwiftUI

The Core Problem: WHO + WHEN + WHAT vs. HOW

✓ The Solved Problems



WHO (Audience Selection)

ML models predict which users have the highest propensity to convert.



WHEN (Send-Time Optimization)

Contextual bandits and history determine when a user is most receptive.



WHAT (Content Ranking)

Recommendation systems match inventory to user preferences.



✗ The Unsolved Problem



HOW (Message Composition)

Knowing the user and context still yields a static, slot-filled template.

```
// Status Quo: Slot-Filling  
"Hey {user.first_name}! Your favorite  
{item.name} is waiting. Order now!"
```

A jarring disconnect: smart targeting delivering a generic message that ignores why the user might act now.

The next evolution in notifications shifts from relevance (finding the right item) to intelligence (composing the right persuasive framing).

The Pizza Example: From Templates to Intelligence

SHARED SYSTEM CONTEXT



John (Late-night user)



11:30 PM



Raining



Pepperoni Pizza



Status Quo (Template)

"Hey John! Your favorite Pepperoni Pizza is waiting for you. Order now!"

WHY IT FAILS

- ✗ Ignores the late-night context
- ✗ Ignores the weather condition
- ✗ Uses generic, pushy CTA ("Order now!")



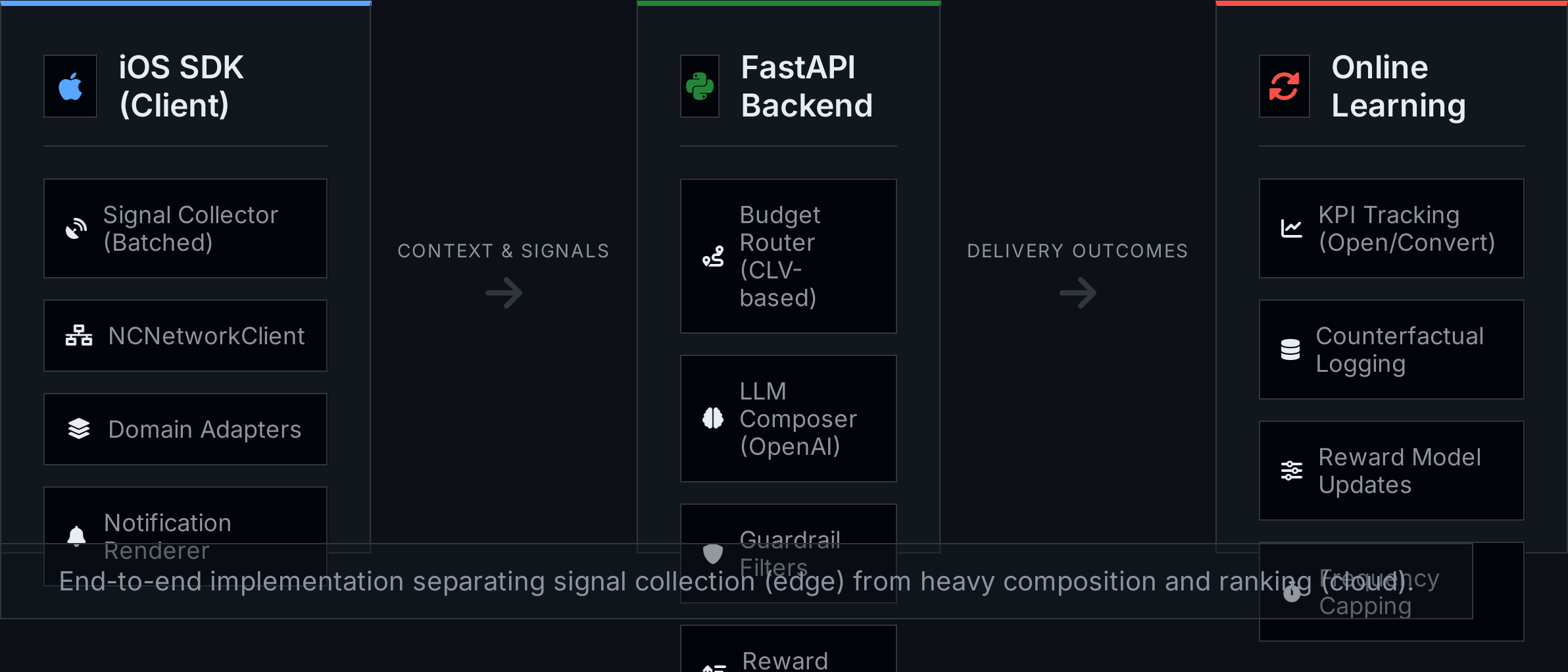
LLM Composition

"Late night studying, John? It's **pouring outside**. Stay dry and let us bring a **hot Pepperoni Pizza** straight to your door."

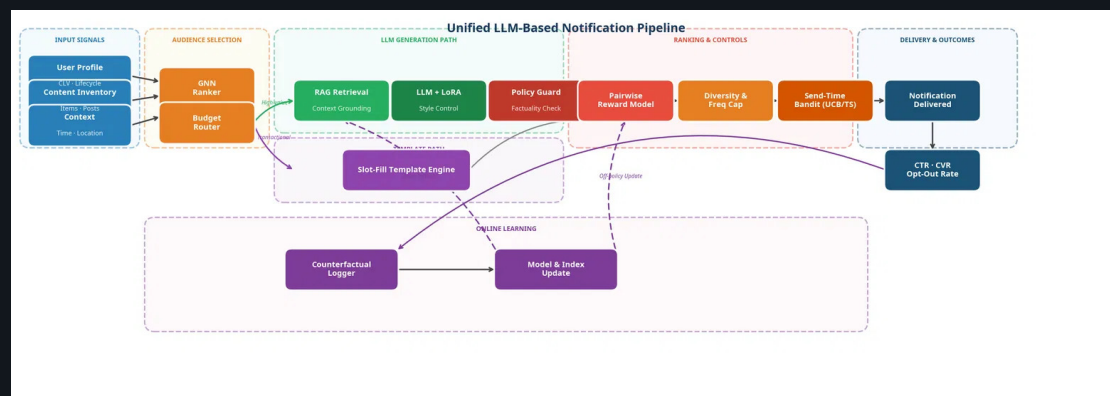
WHY IT SUCCEEDS

- ✓ Synthesizes time (late night) and weather (pouring)
- ✓ Frames the product as a solution to the context
- ✓ Replaces pushy CTA with empathetic framing

Full System Architecture



The FastAPI Backend: 9-Component Pipeline



Budget Router

CLV-tiered routing directing high-value users to LLM generation and others to templates.



Context Retriever (RAG)

Assembles user history, preferences, and real-time signals into a structured prompt.



Message Composer

OpenAI integration generating 5 diverse candidate messages per request.



Guardrail Filter

Blocks hallucinations, dark patterns, and false urgency before ranking.



Reward Ranker

Heuristic and LLM pairwise scoring to select the optimal candidate for delivery.

NotifyCompose iOS SDK (Swift Package)

NotifyComposeClient

The main entry point for the SDK. Provides modern async/await APIs for requesting composed notifications and tracking user interactions.

Automatic Signal Batching

SignalCollector batches user context events and flushes them periodically to optimize battery and network usage.

Domain Adapters

Pre-built adapters (FoodDeliveryAdapter, ECommerceAdapter, SocialMediaAdapter) to construct domain-specific context payloads quickly.

```
let c = NotifyComposeClient.shared;
c.configure(apiKey:"sk-...")
c.track(event:"app_opened", properties:
["time":"night"])
let ctx =
FoodDeliveryAdapter.buildContext(item:"Pizza",
weather:"Rain")
let n = try await c.compose(userId:"user_123",
context:ctx)
c.reportOpened(notificationId:n.id)
```

SwiftUI Sample App: Three Domains



Food Delivery

- > **Scenario:** Lunch/Dinner
- > **Interactive UI:** Weather & time sliders
- > **Demonstrates:** Combine context & prefs

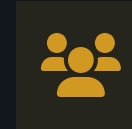
```
FoodDeliveryAdapter.buildContext(weather:  
.raining, time: .lateNight)
```



E-Commerce

- > **Scenario:** Abandoned Cart
- > **Interactive UI:** Product selector & discounts
- > **Demonstrates:** Urgency vs empathy

```
ECommerceAdapter.buildContext(cartValue:  
150, discountActive: true)
```



Social Media

- > **Scenario:** New Content & Activity
- > **Interactive UI:** Activity simulator
- > **Demonstrates:** Summarize social sign

```
SocialMediaAdapter.buildContext(unreadCou  
5, topSignal: .friendPost)
```

Real-Time Pipeline Inspector

Bridging Theory & Practice

The Sample App includes a dedicated Inspector tab that provides full transparency into the SDK's decision-making process.

- ✓ **Paper Mapping:** Every execution step is mapped directly to its corresponding section in the ACM paper.
- ✓ **Traceability:** View the exact context retrieved, candidates generated, and reward scores assigned.

 pipeline_trace.log

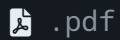
```
11:30:01 [Sec 4.1] BudgetRouter: CLV Tier = Gold -> Routing
to LLM Path
11:30:02 [Sec 4.2] ContextRetriever: Loaded 3 signals
(Weather, Time, History)
11:30:04 [Sec 4.3] MessageComposer: Generated 5 candidate
messages
11:30:05 [Sec 4.4] GuardrailFilter: Candidate 2 REJECTED
(Policy: False Urgency)
11:30:06 [Sec 4.5] RewardRanker: Scoring remaining 4
candidates...
11:30:06 [Sec 4.5] RewardRanker: Candidate 4 selected
(Score: 0.92)
11:30:07 [Sec 4.6] FrequencyCapper: Check passed (1/3 daily
touches)
```


Comprehensive Documentation & Research Paper

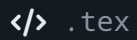


ACM Research Paper

The complete 11-page academic survey and architectural framework, formatted for WSDM '26. Includes all PRISMA methodology, evaluation frameworks, and ethical considerations.



.pdf



.tex



.docx



System Architecture

Detailed technical design document mapping the paper's theoretical components to concrete software modules. Covers signal collection flow and API contracts.



ARCHITECTURE.md



diagrams/



Integration Guide

Comprehensive instructions for running the FastAPI backend locally, integrating the Swift Package into iOS apps, and running the SwiftUI sample application.



README.md



cURL examples



Open-Source License

The entire repository—including the SDK, backend server, sample app, and documentation—is released under the permissive MIT License for academic and commercial use.



MIT License

Conclusion & Open Source Release

💡 Key Takeaways

- ✓ **From Theory to Practice:** The complete ACM paper pipeline is now a fully functional, open-source SDK.
- ✓ **Cross-Domain Ready:** Pre-built adapters for Social Media, Food Delivery, and E-Commerce.
- ✓ **Safe & Scalable:** Built-in guardrails, frequency capping, and CLV-based budget routing.

🔗 Get the Code

github.com/ndagrawal/LLMNotificationComposition

👤 Nilesh Agrawal | ✉️ nilesh.d.agrawal@gmail.com

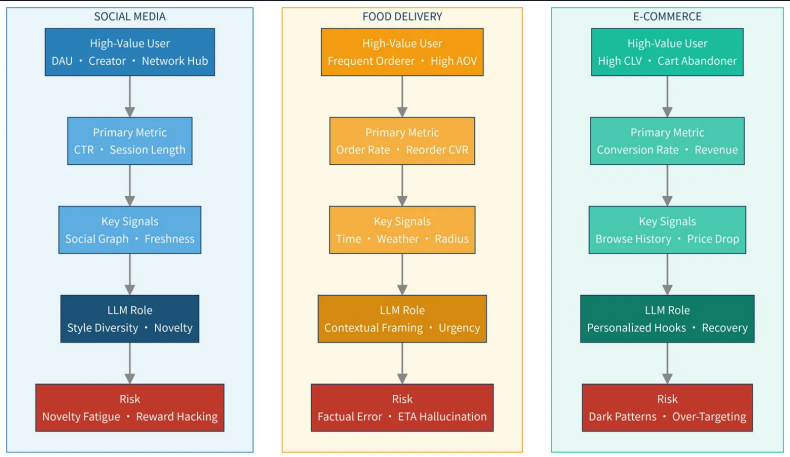


Figure: Domain Generalizability Matrix demonstrating cross-industry applicability.